

Prototype Design Pattern

Intent

Create new objects by **copying an existing instance** (the *prototype*) instead of constructing one from scratch. The caller says "give me another one like this" and the object clones itself — without the caller needing to know its concrete class or how expensive it was to build.

This implementation uses Java's `Cloneable` marker interface and a self-declared `cloneUnit()` method on each product.

UML class diagram

```
<<interface>> IEnemyUnit (extends Cloneable)
+-----+
| +cloneUnit() : IEnemyUnit |
+-----+
      ^
      | implements
+-----+-----+
|      Marauder      |
+-----+-----+
| -armorRating : int  |
| -faction      : String |
+-----+-----+
| +getArmorRating() : int |
| +getFaction()   : String |
| +setFaction(String) |
| +cloneUnit() : IEnemyUnit --> returns new Marauder(armorRating, faction)
| +toString() : String |
+-----+-----+
```

client -> template.cloneUnit() -> independent spawn (equal field values, distinct reference)

The players

prototype/contract/IEnemyUnit
prototype/contract/concrete/Marauder
prototype/EnemySpawnPoint
before/after

the prototype interface (declares cloneUnit())
a concrete prototype that copies itself
the demo – clones a Marauder, mutates the clone, prints both

Code walkthrough

IEnemyUnit — the prototype interface

```
public interface IEnemyUnit extends Cloneable {
    IEnemyUnit cloneUnit();
}
```

- **extends Cloneable** — `Cloneable` is a *marker interface*: it has no methods, it just tags a class as "allowed to be cloned." Its role is explained in *Why Cloneable* below.
- **IEnemyUnit cloneUnit();** — declares the copy operation and gives it a **covariant return type** of `IEnemyUnit`. `Object` already has a protected `Object clone()`; this module deliberately names its own copy operation `cloneUnit()` instead of overriding `Object.clone()` directly, but the intent is identical: it's **public** (so callers outside the class hierarchy can invoke it) and it **narrows the return type** from `Object` to `IEnemyUnit`, so callers get an `IEnemyUnit` back without having to cast the result themselves.

Marauder — a concrete prototype

```
package com.design.patterns.prototype.contract.concrete;

import com.design.patterns.prototype.contract.IEnemyUnit;

public class Marauder implements IEnemyUnit {
    private int armorRating;
```

```

private String faction;

public Marauder(int armorRating, String faction) {
    this.armorRating = armorRating;
    this.faction = faction;
}

public int getArmorRating() {
    return armorRating;
}

public String getFaction() {
    return faction;
}

public void setFaction(String faction) {
    this.faction = faction;
}

@Override
public IEnemyUnit cloneUnit() {
    return new Marauder(armorRating, faction);
}

@Override
public String toString() {
    return "Marauder{armorRating=" + armorRating + ", faction=" + faction + "}";
}
}

```

- **private int armorRating; private String faction;** — the state that must be reproduced in the copy. Both fields are `private`; access is only through the getters and `setFaction`, which is standard encapsulation and keeps `cloneUnit()` as the one place inside the class that reads and copies both fields together.
- **The constructor** is the normal way to build a `Marauder` — but the point of Prototype is to let a caller who *already has* a configured `Marauder` get another one without re-supplying `armorRating` and `faction` (or even knowing them).
- **getArmorRating() / getFaction()** — read-only accessors so external code (like the demo's `toString()`-driven `println` calls) can observe state without needing package access.
- **setFaction(String faction)** — the one mutator. It exists so the demo can prove the clone is independent: a spawned unit changing allegiance must never be visible through the template it was spawned from.
- **cloneUnit()** is the heart of the pattern. It copies **manually**: it reads its own current `armorRating` and `faction` and passes them into a brand-new `Marauder` via the constructor. The returned object is a **separate instance** with equal field values at the moment of copying — not the same reference, and not a re-used, half-initialized object.
- **toString()** overrides `Object.toString()` to print the fields (`Marauder{armorRating=45, faction=Crimson Horde}`) instead of the default `Marauder@<hashcode>`. This makes the demo's output legible and lets it show *value* equality between template and spawn directly, while the separate `!=` check (see below) demonstrates *reference* inequality — two different things that `Object.toString()` alone could not cleanly separate.

EnemySpawnPoint — the demo

```

package com.design.patterns.prototype;

import com.design.patterns.prototype.contract.concrete.Marauder;

public class EnemySpawnPoint {

    public static void main(String[] args) {
        System.out.println("Dungeon Spawn Point");

        Marauder templateMarauder = new Marauder(45, "Crimson Horde");
        Marauder spawnedMarauder = (Marauder) templateMarauder.cloneUnit();

        System.out.println("Template : " + templateMarauder);
        System.out.println("Spawn    : " + spawnedMarauder);
        System.out.println("Distinct objects: " + (templateMarauder != spawnedMarauder));

        spawnedMarauder.setFaction("Ashfall Renegades");
        System.out.println("After the spawn defects:");
        System.out.println("Template : " + templateMarauder);
        System.out.println("Spawn    : " + spawnedMarauder);
    }
}

```

- **new Marauder(45, "Crimson Horde")** — builds the encounter template the normal way, once, up front. Everything downstream avoids repeating this construction.
- **(Marauder) templateMarauder.cloneUnit()** — `cloneUnit()` returns `IEnemyUnit` (the covariant interface type), so the cast narrows it back to `Marauder` to access `Marauder`-specific members like `setFaction`. The spawn is a brand-new `Marauder(45, "Crimson Horde")` built inside `Marauder.cloneUnit()` — the demo never calls `new Marauder(...)` a second time itself.

- `println("Template : " + templateMarauder) / println("Spawn : " + spawnedMarauder)` — because `Marauder` overrides `toString()`, both lines print identical-looking field values (`Marauder{armorRating=45, faction=Crimson Horde}`) right after cloning. This shows the copy reproduced the *state* correctly.
- `templateMarauder != spawnedMarauder` — a reference-identity check. Since value equality alone (matching `toString()` output) could theoretically be produced by returning the same shared reference, this line adds the complementary proof: the two variables point at genuinely different objects in memory. It prints `true`.
- `spawnedMarauder.setFaction("Ashfall Renegades")` — mutates only the spawn (the spawned unit "defects" to a new faction). Because `cloneUnit()` built a *new* `Marauder` with its own `faction` field (not a shared reference to the template's field slot), reassigning the spawn's `faction` cannot touch the template's `faction`.
- **Final two `println` calls** — re-print both objects. `templateMarauder` still shows `Crimson Horde`; `spawnedMarauder` now shows `Ashfall Renegades`. This is the second, complementary proof of independence: not just "different objects" (the `!=` check) but "changing one truly does not affect the other."

Why these design decisions

Why clone instead of just calling the constructor?

Prototype pays off when **construction is expensive or complicated** — the object took a database round-trip, a heavy computation, or a long configuration sequence to reach its current state. Copying an already-built instance skips all of that. It also lets code create new objects **without knowing their concrete class**: given any `IEnemyUnit`, you can call `cloneUnit()` and get another one of the same kind, even if you don't know whether it's a `Marauder`, `Skirmisher`, etc. The prototype instance effectively *is* the factory. This module's `Marauder` is cheap to build, so the win here is purely illustrative — the demo exists to show the mechanics, not to prove a performance case. (In a real spawn point, a template might carry a fully rolled loot table, AI behavior tree, or stat-scaling history that's genuinely expensive to reconstruct per-spawn — cloning the already-configured template is what makes that cheap.)

Why `Cloneable`?

`Cloneable` is Java's opt-in switch for cloning. If a class calls `Object.clone()` **without** implementing `Cloneable`, the JVM throws `CloneNotSupportedException`. So the interface is a safety gate: it signals "this type consents to being cloned." This module declares `IEnemyUnit` extends `Cloneable` to express that intent for the whole family.

Subtlety: this implementation does **not** actually call `super.clone()` — it copies by hand with `new Marauder(armorRating, faction)`. Because of that, it would technically work even without `Cloneable`. Keeping `extends Cloneable` still communicates intent and keeps the door open for implementations that *do* use `Object.clone()`.

Why declare `cloneUnit()` public with an `IEnemyUnit` return type?

- `Object.clone()` is **protected**, so callers in other packages couldn't invoke it. Declaring a fresh copy method on the interface makes it **public**.
- Returning `IEnemyUnit` (not `Object`) is a **covariant return** that spares callers a cast in the common case and keeps the API type-safe.

Manual copy vs. `super.clone()` — and shallow vs. deep

There are two ways to implement a clone operation:

1. **`super.clone()`** — asks `Object` to make a bitwise field-by-field copy. This is a **shallow** copy: primitive fields are duplicated, but object-reference fields are *shared* (both the original and the copy point at the same nested object). Fine for immutable reference fields, but dangerous if a field is a mutable object (a `List`, an array) — mutating it through one copy would affect the other.
2. **Manual copy (what this code does)** — `new Marauder(armorRating, faction)`. You control exactly what gets copied. For a class with mutable nested objects (say, an equipped-item list) you would deep-copy them here (e.g. `new ArrayList<>(other.equippedItems)`), producing a **deep** copy with no shared mutable state.

`Marauder`'s fields are an `int` (copied by value, trivially independent) and a `String` (immutable — even though both objects could safely share the same `String` reference, `setFaction` never mutates the `String` itself, it only *reassigns* the spawn's `faction` field to point at a different `String`). There are no mutable collection or array fields on this class, so shallow vs. deep copy is not actually a live concern here — writing `cloneUnit()` manually simply makes the copy explicit and side-steps the well-known awkwardness of `Object.clone()` (checked exception, `Cloneable` gotchas, no constructor invocation).

Why both a `!=` check and a defect-then-reprint step?

Either proof alone is incomplete. `templateMarauder != spawnedMarauder` proves the spawn is a *distinct object*, but a buggy `cloneUnit()` that shared the same `faction` reference across two otherwise-separate `Marauder` instances would still pass that check. Mutating the spawn via `setFaction("Ashfall Renegades")` and confirming the template is untouched proves the *state itself* is independent, not just the outer object reference. Together they demonstrate what Prototype promises: a full, self-contained copy.

Execution flow trace (as run from `main`)

```
EnemySpawnPoint.main
├── println("Dungeon Spawn Point")           header line
├── new Marauder(45, "Crimson Horde")         build the encounter template
│   └── templateMarauder
├── templateMarauder.cloneUnit()              the object copies ITSELF
│   ├── new Marauder(45, "Crimson Horde")    a brand-new, independent instance
│   └── (Marauder) cast → spawnedMarauder
├── println("Template : " + templateMarauder) → Marauder{armorRating=45, faction=Crimson Horde}
├── println("Spawn   : " + spawnedMarauder)   → Marauder{armorRating=45, faction=Crimson Horde}
├── println("Distinct objects: " + (templateMarauder != spawnedMarauder)) → true
├── spawnedMarauder.setFaction("Ashfall Renegades") mutate ONLY the spawn's faction field
├── println("After the spawn defects:")
├── println("Template : " + templateMarauder) → Marauder{armorRating=45, faction=Crimson Horde}
├── (unchanged)
├── println("Spawn   : " + spawnedMarauder)   → Marauder{armorRating=45, faction=Ashfall Renegades}
└── (only the spawn changed)
```

Expected output

Captured from an actual run (`java -cp target/classes com.design.patterns.prototype.EnemySpawnPoint`):

```
Dungeon Spawn Point
Template : Marauder{armorRating=45, faction=Crimson Horde}
Spawn   : Marauder{armorRating=45, faction=Crimson Horde}
Distinct objects: true
After the spawn defects:
Template : Marauder{armorRating=45, faction=Crimson Horde}
Spawn   : Marauder{armorRating=45, faction=Ashfall Renegades}
```

The process terminates immediately after printing — there is no Spring dependency and no server to keep the JVM alive.

How to run

```
# From the module root directory
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile

JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 java -cp target/classes
com.design.patterns.prototype.EnemySpawnPoint
```

The `-o` (offline) flag works once the parent reactor and dependencies are already in your local Maven cache; drop it on a first build. No additional classpath entries are needed — this module has no runtime dependencies beyond the JDK.

Relationship to the other Creational patterns

- **Factory / Abstract Factory** create objects by **choosing a class** and calling `new` inside the factory.
- **Builder** assembles one complex object **field by field**.
- **Prototype (this module)** skips construction logic entirely and **copies an existing instance**. Reach for it when you already have a fully-configured object and want another just like it, or when you want to decouple "make another one" from the concrete class.

Note: unlike the other modules here, Prototype has no `SpringApplication.run(...)` — it's a plain `main`, because cloning needs no container. It's the purest demonstration of the pattern with nothing else in the way.